

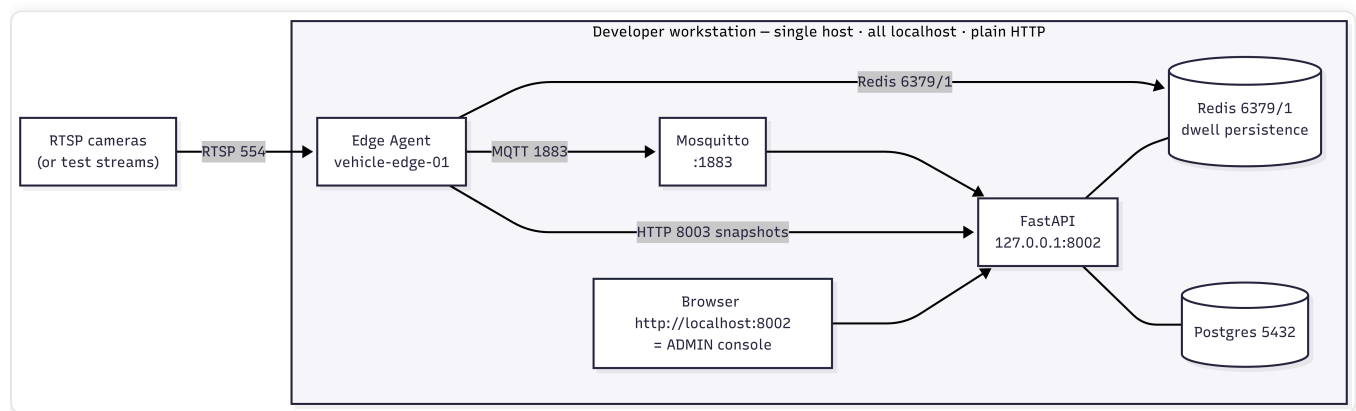
Vehicle Zone Intelligence — Architecture & Deployment Reference

It focuses on four things: the **transition from local to campus network**, the **end-to-end data flow**, **ports & communication protocols**, and the **end-user / application flow**.

1. Transition: Local Setup → Campus Network

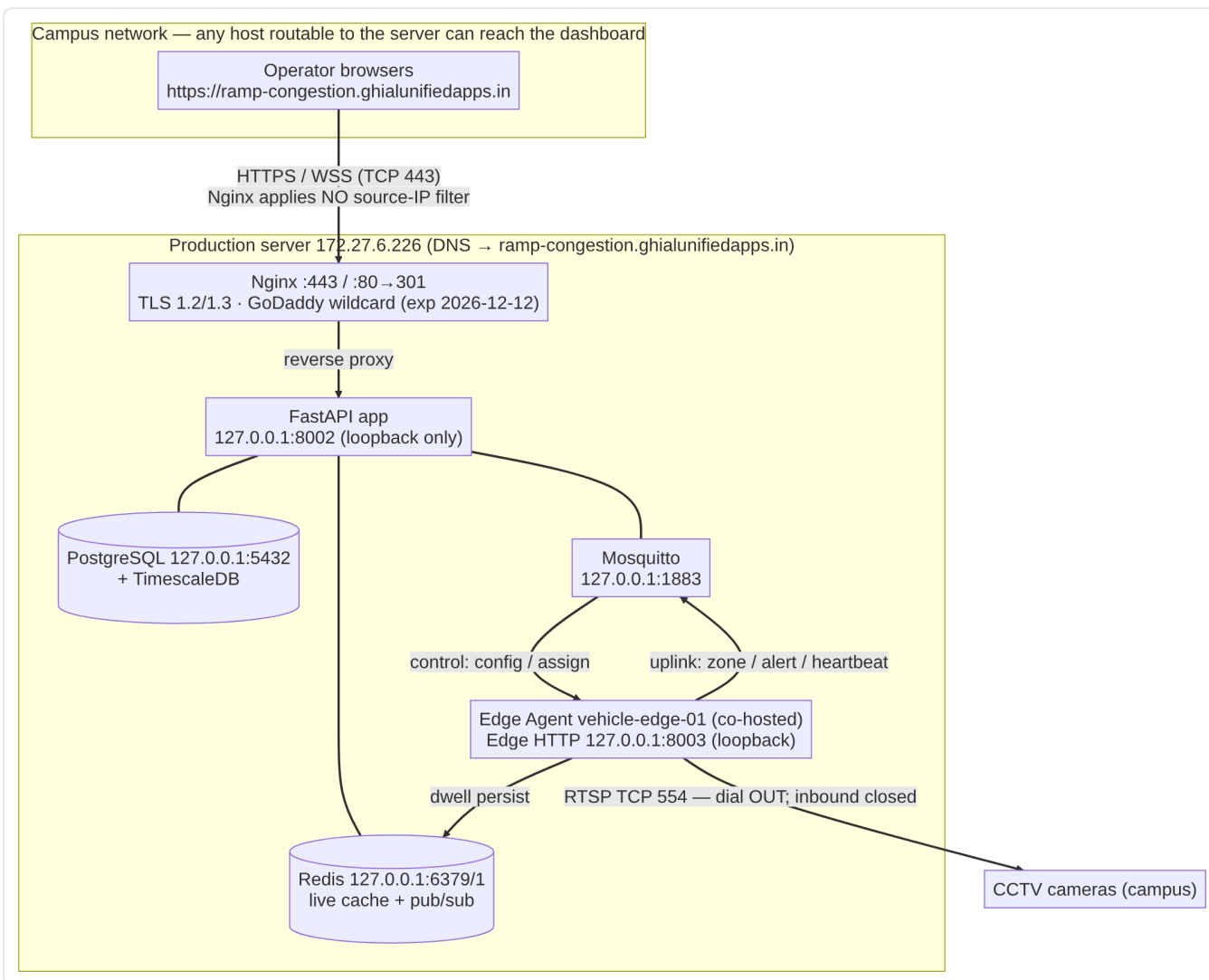
1a. Local / developer setup

Everything binds `localhost` ; plain HTTP; no Nginx, no TLS, no perimeter. The browser reaches the app directly, so it is treated as the **admin** console.



1b. Production campus deployment

Server `172.27.6.226` on the campus network, DNS `ramp-congestion.ghialunifiedapps.in` → `172.27.6.226` . Nginx applies **no source-IP filter**, so the dashboard is reachable by **every host that can route to the server** across the campus network — it is not scoped to any subnet at the application layer.



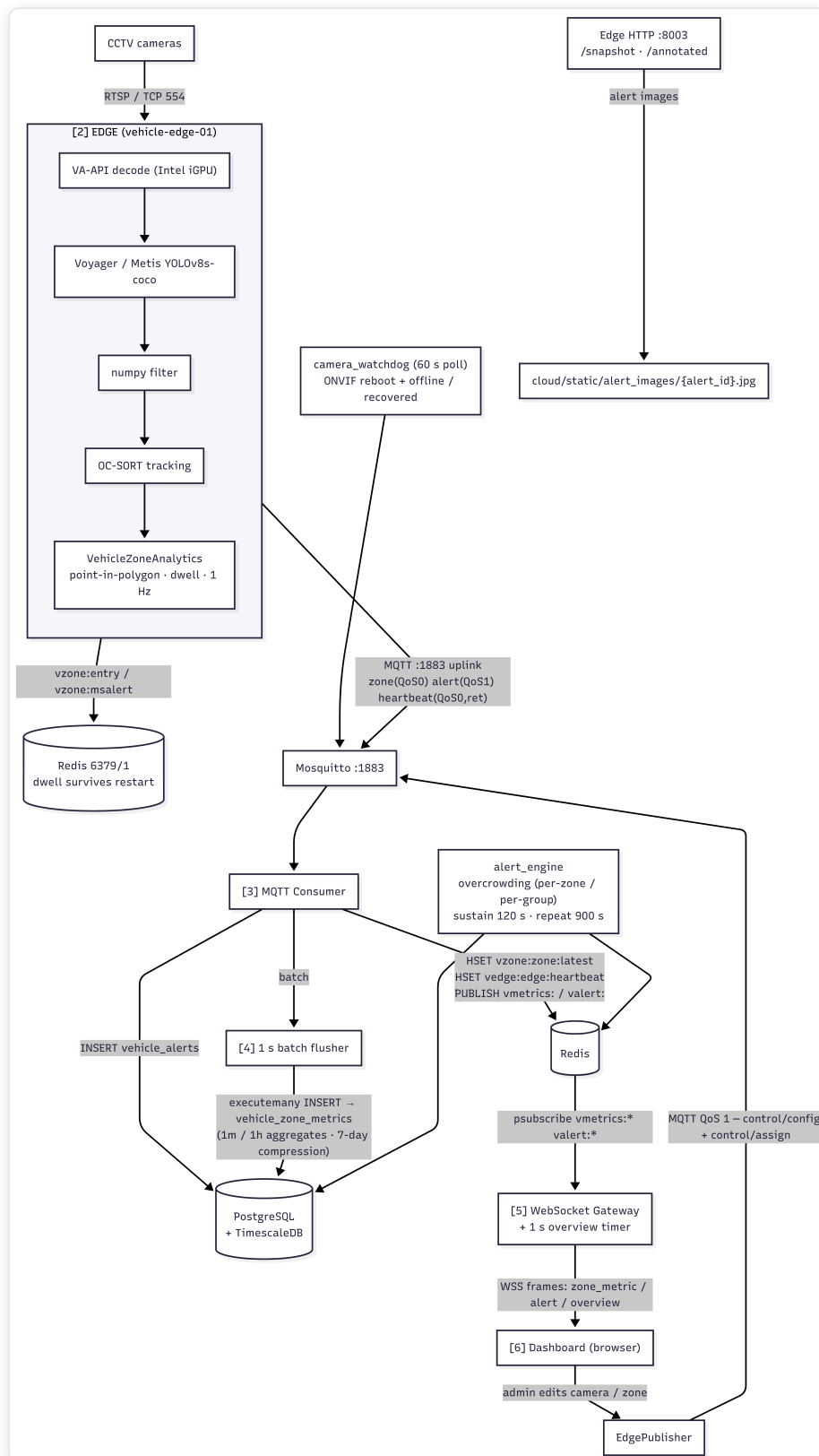
In the current deployment the edge agent, Mosquitto broker, and Redis are all **co-hosted** on 172.27.6.226 , so MQTT and Redis are loopback hops rather than network crossings. The edge is drawn as a distinct component so a future split (edge near cameras, broker on server over MQTT/TLS :8883) is a config change, not a redesign.

1c. What changes local → campus

Concern	Local	Campus production
Front door	app on <code>:8002</code> , plain HTTP	Nginx HTTPS <code>:443</code> , app on <code>127.0.0.1:8002</code>
TLS	none	TLS 1.2/1.3, wildcard cert, HSTS + CSP + security headers
Network reach	all localhost	any host routable to the server on the campus network — not scoped to a subnet by the app
Source-IP masking	n/a	none in effect — Nginx <code>allow / deny</code> is commented out; the optional <code>ufw-setup.sh</code> is the only thing that could add one
Backend ports	n/a	<code>8002/8003/5432/6379/1883</code> bound to loopback — unreachable from the network
DNS	n/a	<code>ramp-congestion.ghialunifiedapps.in</code> → <code>172.27.6.226</code>
CORS	<code>localhost:8002</code>	<code>https://ramp-congestion.ghialunifiedapps.in</code>
Access roles	everyone = admin	console = admin; LAN via Nginx = read-only viewer

Access-boundary caveat: the application does not filter by source IP — Nginx serves every client that can reach it. Whatever bounds access is the network itself (routing, plus the *optional* UFW script), not the app, so treat the dashboard as reachable by any host on the campus network. That exposure is **read-only viewer** access only: mutating actions are gated to the server console (loopback) by `AdminOnlyMiddleware` , and the backend services (DB, Redis, MQTT, edge HTTP) are bound to loopback, so they are never reachable from the network regardless of the firewall.

2. End-to-End Data Flow



Alert ownership (who inserts into `vehicle_alerts`)

Alert type	Originates	Persisted by
<code>overstay</code>	edge analytics (escalating milestone ladder)	MQTT consumer on ingest
<code>overcrowding</code>	cloud <code>alert_engine</code> (per-zone & per-zone-group, Redis live state)	<code>alert_engine</code>
<code>camera_offline</code> / <code>camera_recovered</code>	cloud <code>camera_watchdog</code>	MQTT consumer (watchdog publishes to the edge alert topic)

3. Ports & Communication Protocols

Port	Service	Protocol	Bind / exposure	Direction
443	Dashboard (Nginx)	HTTPS + WSS, TLS 1.2/1.3	no source-IP filter at Nginx — reachable by any host that can route to the server on the campus network	operators → server
80	Nginx	HTTP → 301 redirect to 443	no source-IP filter — campus network	inbound
554	CCTV cameras	RTSP over TCP	server dials out ; inbound closed	edge → cameras
8002	FastAPI app	HTTP (REST + WS upstream)	127.0.0.1 loopback	Nginx → app
8003	Edge HTTP	HTTP (snapshot / annotated / stream / camera_stats / health)	127.0.0.1 loopback	app → edge
1883	Mosquitto	MQTT — uplink (QoS 0/1, some retained) and downlink control (QoS 1)	127.0.0.1 loopback	edge ↔ cloud
6379	Redis (DB 1)	RESP — hash cache + pub/sub	127.0.0.1 loopback	edge + cloud
5432	PostgreSQL + TimescaleDB	PostgreSQL wire (asyncpg)	127.0.0.1 loopback	app internal
22	SSH	SSH	campus network	admin
~~8883~~	MQTT/TLS	<i>not implemented — HLD aspirational; reserved for a future edge/broker split</i>	—	—

Loopback bind — verify on the live host. The repo hardens Mosquitto and Redis to loopback (`mosquitto-loopback.conf` → `listener 1883 127.0.0.1`; `redis-hardening.conf` → `bind 127.0.0.1 -:::1`). On the production server 172.27.6.226, Mosquitto/Redis are **co-hosted and shared with ApexEdge**, and those hardening drop-ins are known to conflict (redis 6.0.16 + the ApexEdge listener) — so the live bind for 1883 / 6379 may be broader than loopback. Confirm on the host with `ss -tlnp '( sport = :1883 or sport = :6379 )'` and re-apply the drop-ins if they didn't take. The app, edge HTTP (8003), and PostgreSQL (5432) loopback binds are enforced in code and are not affected.

MQTT topic map

Direction	Topic	QoS	Retained
edge → cloud	vehicle/edge/{edge_id}/zone/{zone_id}	0	no
edge → cloud	vehicle/edge/{edge_id}/alert/{alert_type}	1	no
edge → cloud	vehicle/edge/{edge_id}/heartbeat	0	yes
supervisor → cloud	vehicle/edge/{edge_id}/supervisor/state	1	yes
cloud → edge	vehicle/control/{edge_id}/config	1	no
cloud → edge	vehicle/control/{edge_id}/assign	1	no

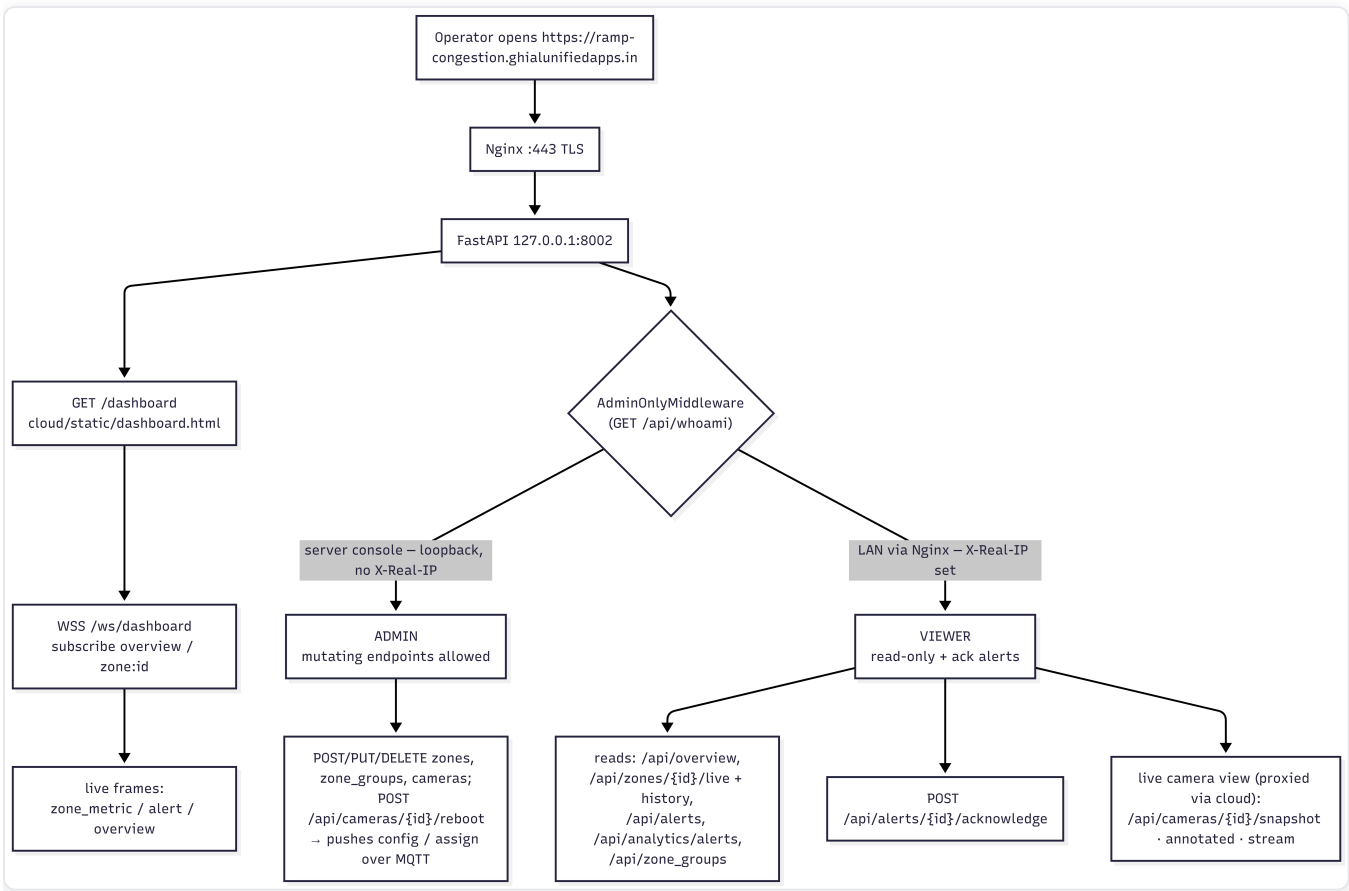
Cloud subscribes (QoS 1) to `vehicle/edge/+/zone/+`, `vehicle/edge/+/heartbeat`, and `vehicle/edge/+/alert/+`.

Redis keys & channels

Kind	Key / channel	Written by
hash	vzone:{zone_id}:latest	MQTT consumer
hash	vedge:{edge_id}:heartbeat	MQTT consumer
key	vzone:entry:{zone}:{track}, vzone:msalert:{zone}:{track}	edge (dwell persistence)
keys	camwd:{cam}:fail_since / :last_reboot / :count: {date} / :alerted	camera watchdog
channel	vmetrics:{zone_id}	MQTT consumer
channel	valert:{zone_id}	MQTT consumer, alert_engine, watchdog
channel	vehicle/maintenance/reboot	camera watchdog

WebSocket gateway pattern-subscribes to `vmetrics:*` and `valert:*`.

4. End User / Application Flow



Verified route inventory

- **System:** GET /health, GET /api/whoami, GET / → /dashboard, GET /dashboard, GET /static/alert_images/{file} (path-traversal-guarded), GET /static/*, WS /ws/dashboard
- **Zones:** GET/POST /api/zones, GET/PUT/DELETE /api/zones/{id}, GET /api/zones/{id}/live, GET /api/zones/{id}/history
- **Zone groups:** GET/POST /api/zone_groups, GET/PUT/DELETE /api/zone_groups/{id}, GET /api/zone_groups/{id}/live
- **Cameras:** GET/POST /api/cameras, GET/DELETE /api/cameras/{id}, POST /api/cameras/{id}/reboot, GET /api/cameras/{id}/snapshot|annotated|stream
- **Alerts:** GET /api/alerts, POST /api/alerts/{id}/acknowledge
- **Overview / analytics:** GET /api/overview (includes edge status), GET /api/analytics/alerts
- **No /api/edges** (despite the HLD) — edge status comes from /api/overview.

WebSocket frame types

zone_metric, alert, overview (data frames) plus subscribed, unsubscribed, and error (control replies). The gateway runs a 1 s overview timer and only forwards a zone frame to a client subscribed to that zone.

5. Database Schema (`schema.sql`)

Table	Purpose	Type
<code>edges</code>	Edge device registry	Standard
<code>cameras</code>	RTSP source inventory, <code>assigned_edge</code> FK	Standard
<code>zone_groups</code>	Logical aggregation of zones across cameras	Standard
<code>zones</code>	Polygon, thresholds, <code>camera_id</code> + <code>zone_group_id</code> FKs, <code>ramp_type</code>	Standard
<code>vehicle_zone_metrics</code>	1 Hz edge telemetry	TimescaleDB hypertable (7-day compression)
<code>vehicle_alerts</code>	Overstay / overcrowding / camera events, <code>image_url</code> , ack fields	Standard

Continuous aggregates `vehicle_zone_metrics_1m` and `vehicle_zone_metrics_1h` roll the hypertable up for fast historical queries.

6. Edge Resilience (supervisor — `start.sh`)

`start.sh` launches the cloud server (`uvicorn` on `127.0.0.1:8002`), the edge agent, and a bash supervisor that watches the edge:

- **Hang watchdog** — `90 s` without throughput-log growth → SIGKILL the edge (detects the Voyager wedge where the process is alive but not inferring). `45 s` startup grace after each (re)launch.
- **Crash restart** — on any exit, run Metis cleanup (`axdevice --refresh` , pre-authorized via NOPASSWD sudoers) and relaunch; ~30 s round-trip.
- **Rapid-crash giveup** — under `120 s` uptime counts as a fast crash; after `5` in a row the supervisor gives up and publishes a **retained** MQTT message `vehicle/edge/{edge_id}/supervisor/state` → `{"state":"dead","reason":"rapid_crash_limit","crashes":5,"ts":...}` (QoS 1). Subscribe to that topic to page operators — the giveup state otherwise looks "healthy" from the cloud (the API/WebSocket keep serving cached data).